# The Edit Distance Problem
# UG Project Report
# CS396A, 2019-20 Sem II
# Project Supervisor - Prof. Raghunath Tewari

Ananta Mukherjee
3rd Year Undergraduate
Dept. of Computer Science and Engineering
Indian Institute of Technology Kanpur
mail - anantam@cse.iitk.ac.in

June 13, 2020

## Abstract

The *edit distance* of two strings is a quantitative measure of how similar those two strings are i.e. how many edit operations (with insertion, deletion and substitution of letters as valid edit operations) are needed to convert one string into another.
The classical dynamic programming approach due to Wagner and Fischer enables one to calculate it in $O(n^2)$ time with the string lengths being $n$. In this project, I have mainly studied the problem in an interesting set up where the *edit distance* is upper bounded by a parameter $k << n$. The algorithms proposed in the paper titled "Streaming Algorithms For Computing Edit Distance Without Exploiting Suffix Trees" by Diptarka Chakraborty, Elazar Goldenberg and Michal Koucký have been the main focus of my study. Though the best algorithm due to Landau et al (1998) runs in time $O(n + k^2)$ (which is known to be optimal under the Strong Exponential Time Hypothesis) it uses a tool called *suffix tree* that may not easy and efficient to be implemented in practical scenario. The aforementioned paper gives a similar type algorithm with run time $O(n+k^3)$ which is implementable without building suffix trees and having no dependence of run time on the alphabet size.

## 1 Introduction

The *edit distance* (aka *Levenshtein distance*) [Lev66] is a widely used distance measure between two strings $x$ and $y$ over some well defined alphabet $\Sigma$. With

insertion, deletion and substitution of letters considered as valid edit operations, edit distance between $x$ and $y$, denoted by $\Delta(x, y)$, is defined as the minimum number of edit operations required to change $x$ into $y$. Since it has immense applications in computational biology, text processing, pattern recognition, information retrieval and several other fields, computing $\Delta(x, y)$ efficiently for given $x, y \in \Sigma^n$ has been a point of serious interest for theoretical computer scientists. Sometimes it is also necessary to find the sequence of corresponding edit operations of $\Delta(x, y)$, known as the *alignment* of $x$ into $y$.

Edit distance has some serious practical applications in computational biology. Say, due to mutation, some $A, T, C, G$ bases have been replaced (edited) by some others in a DNA chain. Now, the degree of alteration in the DNA string due to mutation is to be estimated. This is nothing but an edit distance problem. Since DNA strings are really long (almost $10^9$ bases) we need some time and space efficient algorithm for this. Moreover, it is arguably true that the number of edit operations happened there is very small compared to the string lengths. So, instead of attempting the problem in a general set up, taking the assumption of $\Delta(x, y) \leq k << n$ and then developing a strategy for that scenario does make sense.

## 1.1 Previous Advancements

The first approach was a classic dynamic programming approach due to Wagner and Fischer [WF74]. This runs in $O(n^2)$ time and $O(n)$ space. Then Ukkonen modified this approach in the setup of our focus ($\Delta(x, y) \leq k << n$) and gave a better time complexity of $O(nk)$ and space complexity of $O(k)$. The next significant improvement was done in [LMS98] by building suffix tree of $x$ and $y$. This runs in $O(n.min\{\log n, \log \Sigma\} + k^2)$ time and $O(n)$ space ($\Sigma$ is the alphabet). For a finite alphabet the time becomes $O(n + k^2)$ and it is known to be the best under Strong Exponential Time Hypothesis. The problem with this approach is that it requires the suffix tree building procedure which, despite being a linear time algorithm in theory, is really expensive and cumbersome to be implemented in practice. In the paper of our current focus ("Streaming Algorithms For Computing Edit Distance Without Exploiting Suffix Trees" by Diptarka Chakraborty, Elazar Goldenberg and Michal Koucký) the authors have been able to reduce the space complexity of [LMS98] to $O(k)$. Moreover, they have proposed a $O(n + k^3)$ time and $O(k)$ space algorithm that runs without building any suffix tree and hence is simply enough to be implemented in practice.

# 2 Preliminaries

## 2.1 The Classic Dynamic Programming Approach

The dynamic programming approach due to Wagner and Fischer [WF74] solves the *edit distance* problem in $O(n^2)$ time. We keep on filling an $(n+1) \times (n+1)$ matrix $D$ where $D_{i,j}$ denotes the value of $\Delta(x_{1\ldots i}, y_{1,\ldots j})$. Clearly, if $i = 0$ (i.e. the $x$ string is empty) then $D_{i,j} = j$ since all the $j$ entries has to be inserted in order to get $y$ from $x$. Conversely, $j = 0$ gives us $D_{i,j} = i$. Now, we state the recurrence relation to populate other entries of the $D$ matrix.

$$D_{i,j} = \min \begin{cases} D_{i-1,j} + 1 & \text{if } i > 0 \\ D_{i,j-1} + 1 & \text{if } i > 0 \\ D_{i-1,j-1} + \delta_{x_i,y_j} & \text{if } i, j > 0 \end{cases}$$

This is because $x_{1\ldots i}$ can be changed into $y_{1,\ldots j}$ in 3 possible ways. These 3 ways correspond to the 3 cases in the recurrence expression. First, the $x_i\,th$ letter can be deleted with edit cost 1 and then $x_{1\ldots i-1}$ can be changed into $y_{1\ldots j}$. Second, $x_{1\ldots i}$ can be changed into $y_{1\ldots j-1}$ and then paying unit edit cost $y_j\,th$ letter can be inserted. Finally, $x_{1\ldots i-1}$ can be changed into $y_{1\ldots j-1}$ and then we need to pay a unit edit cost of substitution if $x_i$ is different from $y_j$ but if they are the same we need not pay any more. So, $\delta_{x_i,y_j} = 1$ if $x_i \neq y_j$ and otherwise it's 0. Thus $D_{i,j}$ is the minimum among the three aforementioned possibilities. Since, every cell population is a $O(1)$ task the total time required is $O(n^2)$.

This description can be viewed pictorially by a graph called *edit graph*. For given strings $x, y \in \sigma^n$, the vertices of the graphs are represented by all pairs $(i, j)$ where $0 \leq i, j \leq n$ and a complete grid is constructed using those vertices. Now, we need to put some weights on the grid edges. The edge that connect $(i, j)$ with $(i, j+1)$ $(0 \leq i \leq n,\ 0 \leq jn)$ receives a unit weight, similar for the edge connecting $(i, j)$ with $(i+1, j)$ $(0 \leq in,\ 0 \leq j \leq n)$. Now we add diagonal edges. The edge connecting $(i-1, j-1)$ with $(i, j)$ $(0 \leq in,\ 0 \leq jn)$ has weight 1 if $x_{ij}$, else it receives no weight. Now, $D_{i,j}$ correspond to the grid points $(i, j)$ in a very interesting way. $D_{i,j}$ is nothing but the minimum cost of a path from $(0, 0)$ to $(i, j)$ in the *edit graph* (Example shown in fig. 1)

## 2.2 Sketch of the $O(n + k^2)$ Time Algorithm

Now if we consider the problem in the set up where the *edit distance* is actually upper bounded by some parameter $k << n$, the dynamic programming approach runs successfully in $O(nk)$ time. Ukkonen [UK85] first showed it by realising that we need not fill up the entire matrix $D$ in this scenario. It's sufficient to only fill up the entries between the $-k\,th$ diagonal and $k\,th$ diagonal of the matrix. Here by $d$th diagonal we denote all the points $(i, j)$s where $j = i+d$. So, the principal diagonal is the 0th diagonal. Since, there are $O(nk)$ many en-

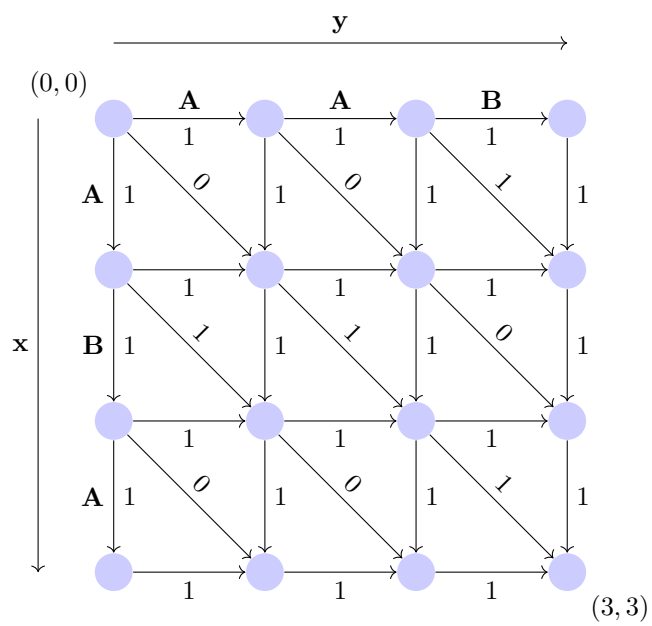


Figure 1: An example of an *edit graph* for strings x and y $\in \{A, B\}^3$

tries lying between $-k\ th$ and $k\ th$ diagonal $O(nk)$ time suffices.

Now we are going to define a key notation $F^h(d)$ that will play a crucial role in our further discussion. For $h \in \{0, 1, 2, ..., n\}$ and $d \in \{-h, ...h\}$ we define $F^h(d)$ as the furthest row that can be reached in the diagonal $d$ with consuming exactly $h$ edit operations. Formally,

$$F^h(d) = max\{i \ : \ D_{i,i+d} = h\}$$

Thus evaluating the values of $F^h(d)$ fully determines the matrix $D$. Moreover, since we are focusing only on the matrix entries between the $-k\ th$ and $k\ th$ diagonal, enumeration of the values of $F^h(d)$ for $h \in \{-k, ...k\}$ and $d \in \{h, ...h\}$ suffices. So, number of $F^h(d)$ values to be evaluated are of $O(k^2)$. In the paper [LMS98], they built a suffix tree of the twos strings (consumes time $O(n)$) and utilising that evaluates each of the $F^h(d)$ values in $O(1)$ time. So, the problem is solved in $O(n + k^2)$ time. The details is necessary for the main focus of this project and is discussed below.

First, we define another important notation $slide_{x,y}(d, i)$. This is defined as the furthest row $q \geq i$ that can be reached on the diagonal $d$ starting from row $i$ on the same diagonal without using any extra edit cost. Formally,

$$slide_{x,y}(d, i) = max\{q \ : i \leq q \leq min\{n, n - d\} \ \& \ x_{i+1,...q} = y_{i+1+d,...,q+d}\}$$

Actually, slide gives a handle to compare the strings $x$ and $y$ with an offset $d$, to be specific it says if $x$ is viewed from $x_i\,th$ position and $y$ is viewed from $y_{i+d}\,th$ position, how long they resemble. $F^h(d)$ also signifies that $h$ is the minimum edit cost required to change $x_{1,\ldots F^h(d)}$ into $y_{1,\ldots F^h(d)+d}$. This slide enables us to give a recurrence relation to evaluate $F^h(d)$ entries.

$$F^h(d) = slide_{x,y}(d, maxS_{d,h})$$

where $S_{d,h}$ is the set stated below

$$\begin{cases} F^{h-1}(d)+1 & \text{if } |d| \leq h-1 \\ F^{h-1}(d+1)+1 & \text{if } d+1 \leq h-1 \\ F^{h-1}(d-1) & \text{if } d-1 \geq -(h-1) \end{cases}$$

The reasoning behind this recurrence is that $x_{1,\ldots F^h(d)}$ can be changed into $y_{1,\ldots F^h(d)+d}$ in 3 possible ways.
First, change use $h-1$ cost to change $x_{1,\ldots F^{h-1}(d)}$ into $y_{1,\ldots F^{h-1}(d)+d}$ optimally. Then use one additional cost to substitute one letter and then slide along diagonal $d$ (no more cost).
Second, use $h-1$ cost to change $x_{1,\ldots F^{h-1}(d+1)}$ into $y_{1,\ldots F^{h-1}(d+1)+d+1}$ optimally. Then add the $F^{h-1}(d+1)+1\,th$ letter in $x$. This will bring it on diagonal $d$ from diagonal $d+1$ and then slide optimally on diagonal $d$.
Finally,use $h-1$ cost to change $x_{1,\ldots F^{h-1}(d-1)}$ into $y_{1,\ldots F^{h-1}(d-1)+d-1}$ optimally. Then add the $F^{h-1}(d-1)+d\,th$ letter in $y$. This will bring it on diagonal $d$ from diagonal $d-1$ and then slide optimally on diagonal $d$.

We define values $c_{d,h}$ to be the cardinality of the set $S_{d,h}$ because due to the constraints on $h$ and $d$ values ($F^h(d)$ should use the values already enumerated, should not call itself.). So, $c_{d,h} \in \{,1,2,3\}$. $c_{0,0}$ is taken to be 1. Now we define $h$ wave as the set of points $F^h(-h), ..., F^h(0), ..., F^h(h)$. We start from computing the 0 wave and then proceed to 1 wave and so on. Whenever some $e$ wave is attained such that $F^e(0) = n$, we stop and $\Delta(x, y)$ becomes $e$.

Now [LMS98] actually shows how to compute these $slide_{x,y}(d, i)$ values in $O(1)$ time. If this is done then $F^h(d)$ values can also be enumerated in $O(1)$ time by using the recurrence. But this requires the suffix tree building procedure to be implemented between $x$ and $y$. They actually build the generalised suffix tree for the string $x\$y\#$ where \$ and # do not belong to the alphabet $\Sigma$. An additional data structure is required to answer the lowest common ancestor in this suffix tree in $O(1)$ time.
This suffix tree building procedure requires $O(n)$ time and $O(n)$ space. Despite being a linear time algorithm in theory, due to the hidden constants and some other complications it actually doesn't help much and often Ukkonen's $O(nk)$ time algorithm is preferred to this. Space wise, the strings $x$ and $y$ can be divided into chunks and we can make it run in $O(k)$ space as well (details discussed in section 4). If we store the values $F^h(d)$ for $|d| \leq h \leq k$ we can compute the

sequence of optimal edit operations as well (optimal alignment). This storage requires $O(k^2)$ space.

# 3 Details of the $O(n+k^2)$ Time Streaming Algorithm

Now we shall discuss the details of implementation of the [LMS98] algorithm and gradually build the pseudo code. We need to evaluate $F^h(d)$ values for $|d| \leq h \leq k$ with the parameter being provided. According to the recurrence relation, In order to get $F^h(d)$ we first need to know $F^{h-1}(d)$, $F^{h-1}(d+1)$ and $F^{h-1}(d-1)$ and then from the greatest among them we compute a slide.
First we introduce $n+1$ lists $L_0, L_1, ..., L_n$. These lists contain the ordered pairs of the form $(d, h)$. If some $(d, h)$ belong to some $L_i$ that indicates that the slide to compute $F^h(d)$ should possibly start from the row $i$ i.e. $i$ is one of the values among $F^{h-1}(d)$, $F^{h-1}(d+1)$ and $F^{h-1}(d-1)$. This implies that at a given instant $(d, h)$ can be the member of a unique list among those $n+1$ ones available. A 2-D $(2k+1) \times (k+1)$ array $D$ is used to point out the list that contains $(d, h)$ currently for every $(d, h)$ pair. Another 2-D array $C$ of same order is used to count the number of times $(d, h)$ has occurred in the lists i.e. the number of options for the rows where the slide will begin for $F^h(d)$ computation, discovered so far.
Additionally, we take another $(2k+1) \times (k+1)$ 2-D array $L^h(d)$ to store the values of corresponding $F^h(d)$s. A suffix tree is also built with the aforementioned string $x\$y\#$ and another spacial data structure is designed to answer the least common ancestor query in that suffix tree in $O(1)$ time.
We start the algorithm by putting $(0, 0)$ in the list $L_0$ and perform a slide along diagonal 0 starting from row 0. Now, if this slide terminates at row, say $q$ then we first update $L^0(0) = q$ followed by adding $(1, 1)$ into $L_q$ and (0,1) and (-1,1) in $L_{q+1}$. Now for every row $i = 0, 1, ..., n$ list $L_i$s are checked. for every entry $(d, h)$ in a list we check whether all the required values for starting slide computation of $F^h(d)$ have been discovered. If yes we compute the slide. If it ends at row $k$ we similarly update $L^h(d) = q$ followed by adding $(d+1, h+1)$ in $L_q$ and $(d, h+1)$ and $(d-1, h+1)$ in $L_{q+1}$.
The detailed pseudo code is given below, the pseudo code is taken from the paper of our focus i.e. "Streaming Algorithms For Computing Edit Distance Without Exploiting Suffix Trees" by Diptarka Chakraborty, Elazar Goldenberg and Michal Koucký

**Algorithm 1:** The $O(n + k^2)$ time [LMS98] algorithm

```
Input: x, y ∈ Σ^n and a parameter k ≤ n such that Δ(x, y) ≤ k
Output: Δ(x, y)
//initialization
Build generalised suffix tree of x and y to calculate slide_{x,y}(d, i) queries
  in O(1) time.
for i = 0, 1, ...n initialize vacant lists L_i, for all (d, h) with |d| ≤ h ≤ k
  set D(d, h) = null and C(d, h) = 0
Invoke Update(0, 0, 0);
for i = 0, 1, ..., n do
    while L_i is not empty do
        Pick (d, h) from L_i and remove it from list;
        if C(d, h) = c_{d,h} then
            q = slide_{x,y}(d, i);
            L^h(d) = q;
            if h < k then
                Update(d, q + 1, h + 1);
                if d < k then
                    Update(d, q + 1, h + 1);
                end
                if d > −k then
                    Update(d, q + 1, h + 1);
                end
            end
        end
    end
end
Output the smallest h ≤ k such that L^h(0) = n.
```

**Procedure** *Update*(d,i,h)

```
Increase C(d, h) by one.
if D(d, h) = null or D(d, h) points to an entry in L_{i'} then
    if D(d, h) ≠ null then
        Delete the current node pointed to by D(d, h) in L_{i'};
    Add (d, h) at the back of L_i;
    Set D(d, h) to point to the new entry (d, h);
```

## 3.1 Complexity

Time complexity of the aforementioned algorithm majorly depends on two things. First is the construction of the generalised suffix tree and the data structure required for efficient computation of lowest common ancestors. This requires $O(n.min\{\log n, \log \Sigma\})$ [Gus97]. So, for finite alphabet size this be-

comes $O(n)$. Secondly, the lists $L_i$s have to be processed for $i = 0, 1, ..., n$. Now whenever a $(d, h)$ pair is handled and deleted from a list it never reappears. Moreover, due to the efficient slide computation every $(d, h)$ pair handling takes $O(1)$ time. Since, $|d| \leq h \leq k$, we need to handle $O(k^2)$ many such pairs. So, total time complexity is of $O(n.min\{\log n, \log \Sigma\}) + k^2$ i.e. for fixed alphabet size $O(n + k^2)$.

Now, we discuss the space requirement. The suffix tree building procedure takes $O(n)$ space. There are $n + 1$ many lists containing $(d, h)$ pairs. Since, $|d| \leq h \leq k$ and no $(d, h)$ pair appears more than once we require $O(n + k^2)$ many space. The storage of arrays $D$, $C$ $L^h(d)$ also requires $O(k^2)$ many space for the same constraints on $d$, $h$ and $k$. So, total space complexity is $O(n + k^2)$.

# 4 The Suffix Tree Algorithm in $O(k)$ Space

One bottleneck of the aforementioned algorithm was it consumed $O(n)$ (way bigger than $O(k)$ and $O(k^2)$ and may be difficult to implement for practical scenario) space to build the suffix tree as a prerequisite to calculate the slides efficiently. To eliminate this problem we divide the strings $x$ and $y$ in blocks of length $O(k)$ (so. $O(\frac{n}{k})$ such parts will be there) and process them by building suffix trees block by block)
For each block, the suffix tree enables us to calculate slide within that block in $O(1)$ time. Instead of building and maintaining lists $L_0, L_1, ..., L_n$, here we only consider $k + 1$ many lists $L_0, L_1, ..., L_k$. While processing the $j\,th$ block we actually consider slide between the rows $jk$ and $(j + 1)k - 1$ ion the setting.
Now, one important thing is slides are not guaranteed to be contained inside a block i.e. a slide can start from one block and may continue beyond it. Here, the lists $L_0$ and $L_k$ play a special role. For every block the list $L_0$ contain the slides that are continuing from the previous block and $L_k$ holds the slides that will continue beyond the block under processing.
The $j\,th$ block of $x$ consists of $x_{jk+1,...,(j+3)k}$ and that of y consists of $y_{(j-1}k + 1, ..., (j + 2)k$. This actually creates the scenario such that slides along every diagonal between $-k$ and $k$ (including them) on the rows between $jk$ and $(j + 1)k - 1$ in the original set up can be computed by calculating the slides on $x'$ and $y'$ in the blocks. We also have diagonal $d$ of the original edit distance matrix between $x$ and $y$ to be the same to the diagonal $d + k$ in the $x'$, $y'$ set up.
Now, we state the pseudo code of the algorithm obtained from the paper of our focus, the procedure *Update* remains the same.

**Algorithm 2:** The Block Divided Suffix Tree Algorithm

```
Input: x, y ∈ Σ^n and a parameter k ≤ n such that Δ(x, y) ≤ k
Output: Δ(x, y)
for i = 0, 1, ...n initialize vacant lists L_i, for all (d, h) with |d| ≤ h ≤ k
  set D(d, h) = null and C(d, h) = 0
Invoke Update(0, 0, 0);
for j = 0, 1, ..., ⌈n/k⌉ − 1 do
    Let, x' = x_{jk+1,...,(j+3)k+1} and y' = y_{(j−1)k+1,...,(j+2)k+1};
    if j=0 then
        y' = 0^k.y';
    end
    Build generalised suffix tree of x' and y' to calculate slide_{x',y'}(d, i)
     queries in O(1) time.
    for i = 0, 1, ..., k − 1 do
        while L_i is not empty do
            Pick (d, h) from L_i and remove it from list;
            if C(d, h) = c_{d,h} then
                q = slide_{x',y'}(k + d, i);
                if q ≥ k then
                    //Partial slide because the slide will continue to the
                     next block
                    Insert (d, h) into the list L_k;
                    Set D(d, h) pointing to the new entry (d, h);
                end
                else
                    L^h(d) = q + jk;
                    if h < k then
                        Update(d, q + 1, h + 1);
                        if d < k then
                            Update(d, q + 1, h + 1);
                        end
                        if d > −k then
                            Update(d, q + 1, h + 1);
                        end
                    end
                end
            end
        end
    end
end
Output the smallest h ≤ k such that L^h(0) = n.
```

### 4.1 Complexity

**Claim:** In every iteration $j = 0, 1, ..., \lceil \frac{n}{k} \rceil - 1$, time required in bounded by $O(k.min\{\log k, \log \Sigma\}) + k_j$, where $\sum k_j = O(k^2)$.
**Proof:** In each $j$ we build suffix trees between $x'$ and $y'$ both having length $O(k)$. This requires $O(k.min\{\log k, \log \Sigma\})$ time. Apart from this, the task performed in each loop is the exact same i.e. handling the $(d, h)$ pairs each in time $O(1)$ by using the effecting slide computing. A $(d, h)$ slide between the rows $jk$ and $(j + 1)k - 1$ in the original setting is handled in the $jth$ loop here between rows 0 and $k$. Since, total $O(k^2)$ many such $(d, k)$ pairs were there in the original setting (since $|d| \leq h \leq k$), here also the the $k_j$ $s$ should some up to $O(k^2)$. But one thing has to be highlighted. Since one slide can continue from one block to another a $(d, h)$ pair can be deleted from one list and get added into another. But since for a given $j$, $L_0$ can contain at max $2k + 1$ items (different options for $d$) and the same item can be added to any list at most thrice, we get the required bound.

Summing over the time $O(k.min\{\log k, \log \Sigma\}) + k_j$ over all $j$ $s$ i.e. $j = 0, 1, ..., \lceil \frac{n}{k} \rceil - 1$, we get the total time $O(n + k^2)$ for fixed alphabet size.
Regarding space complexity, the suffix tree building at each step requires $O(k)$ space. Once a loop of $j$ is over, the previous suffix tree need not be stored. Moreover, the list manipulations require $O(k^2)$ space in total. So, $O(k^2)$ is sufficient for the entire algorithm.
Now, total space can also be diminished to $O(k)$ as well if we only want to compute the edit distance barring the optimal alignment. For a given diagonal $d$, if $h$ is maximal such that $L^h(d)$ was already set then the lists $L_0, L_1, ..., L_n$ can contain entries of the form $(d, h')$ where $h' \in \{h+1, h+2\}$. This also means that arrays $C$ and $D$ also have only $O(1)$ relevant entries for every $d$. So, than can be replaced by effective data structures of size $O(k)$ with lookup and update time of $O(1)$. This strategy is also applicable for the algorithm 2 i.e. block wise processing. Since building of suffix tree and handling of arrays and lists, both can be done in $O(k)$ space so overall space complexity is reduced to $O(k)$.

## 5 Edit Distance Computing Without Using Suffix Trees : The $O(n + k^3)$ Time Algorithm

The suffix tree building procedure may require only linear time theoretically but practically there are quite expensive and cumbersome to build. Hence, for practical scenario mainly the $O(nk)$ algorithm by Ukkonen [UK85] is used. Here, in the paper of our current focus he authors have proposed an $O(n + k^3)$ time algorithm that will not require any suffix tree at all and can be implemented in practice for its simplicity.
The key observation is that we need not process the slides individually i.e. we can group them and process a group as a whole by checking some key behaviour of that group. In particular, if we slide along two different diagonals $d$ and $d'$

with $d < d'$ for more than $2(d' - d)$ common rows then the corresponding two parts of $x$ and $y$ are actually periodic with period $d' - d$. So, instead of sliding separately on them we can just slide along diagonal $d'$ and keep on checking this periodicity property. Whenever the check fails we can be sure that at least one of those 2 diagonals has ended its slide and then one edit operation can be used.
The aforementioned concept holds for a group of diagonals as well. If we slide along the diagonals $d_1, d_2, ..., d_l$ with $d_1 < d - 2 < ... < d_l$ in parallel for more than $2(d_l - d_1)$ common rows then corresponding parts are periodic with periodicity $gcd(d_l - d_1, d_l - d_2, ..., d_l - d_{l-1})$. So, we can slide along the rightmost diagonal (the diagonal $d_l$) only and keep on tracking this periodicity. Whenever this periodicity ends we are supposed to pay an edit operation to either the rightmost diagonal only or everyone barring the rightmost or everyone.
As stated above for a set of of diagonals $2(d_l - d_1)$ row resemblances are necessary to guarantee periodicity. This is because if $w \in \Sigma^*$ is a string and $w'$ is a prefix of $w$ then we don't consider $ww'$ as periodic here i.e. at least $ww...$ is necessary for periodicity to be guaranteed. For set diagonals $\in \{-k, ..., k\}$, $d_l - d_1 \leq 2k$. So, if any of set of diagonals show resemblance for more than $4k$ many rows we can be sure of the periodic nature. As the average length of slide $n/k >> 4k$ for $k << \sqrt{n}$, this approach gives a significant advantage.

Now we will details of the algorithm. First, we state the notion of *maturity* of diagonals. We say that a diagonal d is *mature* at row $i$ iff $x_{i-4k+1,...,i} = y_{i-4k+1+d,...,i+d}$. So, from our previous discussion if we have a set of diagonals where everyone is *mature* at row $i$ then we know that the corresponding parts of $x$ and $y$ are periodic. Similar to our previous algorithm, we maintain $n + 1$ many lists $L_0, L_1, ..., L_n$ out of which at a given time only two consecutive ones can be non empty. We also keep track when a $(d, h)$ pair starts sliding. So, inside the lists we actually insert triplets of the form $(d, h, startSlide)$. If a diagonal slides more than $4k$ steps we delete it from the corresponding list and insert into $matureList$.
The procedure $processMatureDiagonals$ maintain the length of current period of $x$ and $y$ inside a variable $maturePeriod$ and $rightMature$ stores the index of the rightmost diagonal in $matureList$. Apart from that we maintain the lists $D$ and $C$ in a similar fashion.
The algorithm processes the edit distance matrix row by row. At row $i$ it does majorly two things. First it deals with the mature diagonals discovered so far. It checks whether the resemblance between $x$ and $y$ strings (i.e the slide) is maintained across the rightmost diagonal and whether the periodicity property is satisfied in the $x$ string (if yes then it will guarantee slide in every other diagonal barring the rightmost). Thus it decides in which diagonals slides should be carried forward and which diagonals should be deleted from $matureList$ and replaced into the normal lists by paying edit cost.
Secondly, the algorithm processes the entries in the current list $L_i$. If all the options for starting the slide in a certain $(d, h)$ have been discovered already (i.e. if $C(d, h) = c_{d,h}$ then only it moves forward. If the slide continues then

it is checked whether it is eligible to be entered into the *matureList* (i.e. if $i - startSlide > 4k$). If yes, then such is done. If no, then $(d, h, startSlide)$ is moved into $L_{i+1}$. Now, if slide ends at row $i$ (i.e. $x_{i+1} \neq y_{i+1+d}$ then the necessary updates are done.
There are two key concepts. One is related to maintaining the period size of the *matureList* (5.2.1) and the another deals with how to process the *mature* diagonals i.e. how the periodicity property actually ensures the slide and vice versa (5.2.2). We first state the pseudo code as given in the paper of our focus and after that we will discuss them in the correctness section (5.2) in details.

**Algorithm 3:** The $O(n + k^3)$ time [LMS98] algorithm

```
Input: x, y ∈ Σ^n and a parameter k ≤ n such that Δ(x, y) ≤ k
Output: Δ(x, y)
for i = 0, 1, ...n initialize vacant lists L_i, initialize matureList to be
  empty, for all (d, h) with |d| ≤ h ≤ k set D(d, h) = null and
  C(d, h) = 0
Set rightMature = ∞ and maturePeriod = 0;
Invoke Update(0, 0, 0);
for i = 0, 1, ..., n do
    processMatureDiagonals(i);
    while L_i is not empty do
        Pick (d, h) from L_i and remove it from list;
        if C(d, h) = c_{d,h} then
            if x_{i+1} ≠ y_{i+d+1} or i + d + 1 > n then
                L^h(d) = i;
                if h < k then
                    Update(d, q + 1, h + 1);
                    if d < k then
                        Update(d, q + 1, h + 1);
                    end
                    if d > −k then
                        Update(d, q + 1, h + 1);
                    end
                end
                else
                    if i − startSlide > 4k then
                        moveToMatureDiagonals(d, h, startSlide);
                    end
                    else
                        Insert (d, h, startSlide) into L_{i+1};
                    end
                end
            end
        end
    end
end
Output the smallest h ≤ k such that L^h(0) = n.
```

**Procedure** *resetRightMature*(d)

```
Set rightMature = d and maturePeriod = 0;
//Recompute the period
foreach (d', h·startSlide') ∈ matureList do
    if d ≠ d' then
        maturePeriod = gcd(maturePeriod, d − d');
```

**Procedure** *moveToMatureDiagonals*(d,h,startSlide)

```
Add (d, h, startSlide) into matureList and make D(d, h) to point that
  entry;
if matureList has only one entry then
    Set rightMature = d and maturePeriod = 0;
else
    if d > rightMature then
        resetRightMatrure(d);
    else
        maturePeriod = gcd(maturePeriod, rightMature − d);
```

**Procedure** *processMatureDiagonals*(i)

```
if matureList is non − empty then
    if matureList contains only one entry then
        if x_{i+1} ≠ y_{i+1+rightMature} then
            //The unique mature diagonal has a mismatch
            Move the entry of diagonal rightMature from matureList to
              L_i;
        return;
    if x_{i+1} = x_{i+1−maturePeriod} then
        if x_{i+1} ≠ y_{i+1+rightMature} then
            //The rightmost mature diagonal has a mismatch but none
              else
            Move the entry of diagonal rightMature from matureList to
              L_i;
            Find d, the current largest diagonal in matureList and
              invoke resetRightMature(d);
        return;
    //Mismatch on all but possibly the rightmost mature diagonal
    foreach (d, h, startSlide) ∈ matureList do
        if d ≠ rightMature then
            Move the entry (d, h, startSlide) from matureList to L_i;
    if x_{i+1} ≠ y_{i+1+rightMature} then
        //Mismatch also on the rightmost diagonal
        Move the entry of diagonal rightMature from matureList to L_i;
    Set maturePeriod = 0;
```

## 5.1 Propositions

First we will state 3 key facts (with brief justifications) from [CR94]. **Claim 5.1.1:** Let $x \in \{0,1\}^*$ be a string and is periodic with period size p and q.

Then x is periodic with period size $gcd(p, q)$.
**Justification:** WLOG let's assume $p < q$. If $p|q$ then period size is $p$ and we are done. If not, inside the first period of length q, there will be $\lfloor \frac{q}{p} \rfloor$ many complete periods of length p and a chunk of length $q \; mod \; p$. Call this chunk z, the string period with size p to be $w_p$ and that of q to be $w_q$. So, $w_q = w_p.w_p....z = W_p.z$. Now if the entire string is w, then $w = w_p.w_p.... = W_p.w_p... = w_q.w_q... = W_p.z.W_p.z....$ The third and the fifth term gives $z$ is prefix of $w_p$. So, $w_p = z.w_{z1}$ say. Invoking this in the equality chain we get $z$ is also a prefix of $w_{z1}$ i.e. $w_{z1} = z.w_{z2}$ i.e. $w_p = z^2.w_{z2}$. Continuing this we get $w_p$ is periodic with period $z$. So, period size of $w = q = p = q \; mod \; p$ and so on. This, by classic Euclidean lemma of $gcd$, gives period of $w$ is $gcd(p, q)$.

**Claim 5.1.2:** Let $w, u, v \in \{0, 1\}^*$ be such that $vw = wu$ and $|v| = |u| \leq |w|$ . Then $vw$ is periodic with a period of size $|v|$.
**Justification** From the condition we get $v$ is a prefix of $w$ i.e. $w = vw_1$. Invoking this we get $vw = vw_1u \Rightarrow w = w_1u \Rightarrow vw_1 = w_1u$ i.e. $v$ is again a prefix of $w_1$. Continuing this we obtain, $w = v^kv_0$ for some $k \in \mathbb{N}$ and $k_0$ as a prefix of $v$. QED.

**Claim 5.1.3:** Let $w, u, v \in \{0, 1\}^*$ be such that $vw$ and $wu$ are periodic with a period of the same size $\leq |v|, |u| \leq |w|$. Then $vwu$ is also periodic with a period of the same size.
**Justification:** Let the period (not length, the string) is $p$. If $|p|$ divides $|w|$ then we are done because then $u$ starts with $p$ and $vw$ appended $u$ becomes periodic. Otherwise, since $wu$ is periodic, if $w = p.p...z$ (z being a prefix of p), $u$ must begin with the corresponding suffix of $p$ wrt $p$ i.e. $u$ should have a prefix $u_p$ such that $z.u_p = p$. Thus, $vwu$ also becomes periodic with $p$.

## 5.2 Correctness

**Claim 5.2.1:** Let $x, y \in \Sigma^n$ . Let $d' > d \in \{k, ..., k\}$ be diagonals, let $2(d'-d) \leq m \leq i \leq n$. If $x_{i-m+1,...,i} = y_{i-m+1+d,...,i+d}$ and $x_{i-m+1,...,i} = y_{i-m+1+d',...,i+d'}$ Then both $x_{i-m+1,...,i}$ and $y_{i-m+1+d,...,i+d}$ are periodic with a period of size $d' - d$ .
**Proof:** We set $v = y_{i-m+1+d,...,i-m+1+d'-1}, w = y_{i-m+1+d',...,i+d}, u = y_{i+d+1,...,i+d'}$. Now, our claim 2 from subsection 5.1.1 gives the required result.

**Claim 5.2.2:** let $x, y \in \Sigma^n$ and $i$ be a row number with $4k \leq i < n$. Let $M = \{d_1 < d_2 < ... < d_l\}$ be a set of diagonals from $\{-k, ...k\}$ be such that for every diagonal d we have $x_{i-4k+1,...,i} = y_{i-4k+1+d}$. Let $p = gcd(d_l - d_1, d_l - d_2, ..., d_l - d_{l-1})$.Then
1.$x_{i-4k+1,...i}$ and $y_{i-4k+1+d_1,...,i+d_l}$ are periodic with period p.
2.If $x_{i+1} = x_{i+1-p}$ then for all $j \in \{1, 2, ..., l-1\}$, $x_{i+1} = y_{i+1+d_j}$.
3.If $x_{i+1} \neq x_{i+1-p}$ then for all $j \in \{1, 2, ..., l-1\}$, $x_{i+1} \neq y_{i+1+d_j}$.
**Proof:** For the first part, we apply the claim 5.2.1 with $d' = d_l$ and $d = d_j$ for $j = 1, 2, ..., l-1$. Thus, $x_{i-4k+1}$ is periodic with period size $d_l - d_j$ for every

$j = 1, 2, ..., l-1$. So, the claim 5.1.1 gives that $x_{i-4k+1}$ is also periodic with size $p$ and the same is true $y_{i-4k+1+d,...,i+d}$ for all $dM$. Repeating the claim 5.1.3 multiple times we also confirm that $y_{i-4k+1+d_1,...,i+d_l}$ is periodic with the same period size.
Now we concentrate on the second and the third part. Since $y_{i-4k+1+d_1,...,i+d_l}$ is periodic with period size $p$, for all $d_j < d_l$ we get $y_{i+1+d_j} = y_{i+1+d_j-p}$. Due to the diagonal property, $x_{i+1-p} = y_{i+1-p+d_j} \Rightarrow x_{i+1-p} = y_{i+1+d_j}$. Thus, $x_{i+1} = x_{i+1-p}$ iff $x_{i+1} = y_{i+1+d_j}$.

### 5.3 Complexity

First, we consider the $resetRightMature$ procedure. Since every diagonal (out of 2k+1 many of those) can become a $mature$ one at most $k$ times (since number of slides is upper bounded by k), this procedure can be invoked at most $O(k^2)$ many times. If invoked, it computes some gcd of values in the range $\{, 1, ..., 2k\}$ using Euclidean Algorithm. This can be done in $O(k)$ time. So, total time in this process is $O(k^3)$.
Similar argument works for the $moveToMatureDiagonals$ procedure. Now, we deal with the $processMatureDiagonals$ procedure. When we don't remove any diagonal from the $matureList$ it runs in $O(1)$ time. If something needs to be removed we need $O(|matureList| \leq O(k))$ time barring the time already taken care of in the $resetRightMature$ procedure. At least one diagonal is removed from it for at most $O(k^2)$ times. So, the time is upper bounded by $O(k^3)$. When not removing any diagonal, we spend here $O(n)$ time.
In the main loop, any diagonal can continue slide up to $4k$ many steps because after that it will be removed from the lists and will be added in the $matureList$. Since, $2k+1$ ,any diagonals are there and number of slides is upper bounded by $k$ so total time spent in the main loop (barring the time involved in any other procedure) is upper bounded by $O(k^3)$. So, overall the time is bounded by $O(n+k^3)$ and actually it can be written $n + O(k^3)$ i.e. the amount of work needed per one of the n symbols is tiny when not contributing to the $O(k^3)$ time bound.
The space requirement is $O(k)$ and the analysis is the same to that of algorithm 2.

## 6 Possible Future Works

As mentioned above the $O(n+k^2)$ time for the edit distance problem in the set up $\Delta(x, y) \leq k \leq n$ is known to be the best under the Strong Exponential Time Hypothesis. So, one can put effort to solve it $O(n+k^2)$ time but without using suffix trees. Apart from this approximation algorithms can also be effective and interesting i.e. instead of enumerating $\Delta(x, y)$ accurately one may be interested in finding a meaningful approximation of it efficiently.

# References

**Main focus of the project:** "Streaming Algorithms For Computing Edit Distance Without Exploiting Suffix Trees" by Diptarka Chakraborty, Elazar Goldenberg and Michal Koucký

**Other Resources:**

[CR94] Maxime Crochemore and Wojciech Rytter, Text algorithms, Oxford University Press, 1994.

[Gus97] Dan Gusfield, Algorithms on strings, trees, and sequences - computer science and computational biology, Cambridge University Press, 1997.

[Lev66] VI Levenshtein, Binary Codes Capable of Correcting Deletions, Insertions and Reversals, Soviet Physics Doklady 10 (1966), 707.

[LMS98] Gad M. Landau, Eugene W. Myers, and Jeanette P. Schmidt, Incremental string com- parison, SIAM J. Comput. 27 (1998), no. 2, 557–582.

[UK85] Esko Ukkonen, Algorithms for approximate string matching, Inf. Control 64 (1985), no. 1-3, 100–118.

[WF74] Robert A. Wagner and Michael J. Fischer, The string-to-string correction problem, J. ACM 21 (1974), no. 1, 168–173.